

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №3

на тему:

**«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ. КРИПТОСИСТЕМА
РАБИНА»**

БГУИР 6-05-0612-02 74

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Выполнение работы	4
Вывод.....	6
Приложение А (обязательное) Исходный код программы	7

ВВЕДЕНИЕ

Криптосистема Рабина – это криптографическая система с открытым ключом. Её стойкость основана на сложности извлечения квадратных корней в кольце вычетов по модулю составного числа, то есть на задаче разложения большого числа на простые множители, как и у метода RSA. Особенность системы в том, что зашифрованное сообщение имеет четыре квадратных корня, и любой из них может оказаться исходным текстом. Поэтому при расшифровании необходимо отличать истинное сообщение от трёх ложных. В разработанной программе эта проблема решена добавлением избыточности в каждый блок открытого текста: блок начинается с маркера «RBN1» и заканчивается контрольной суммой CRC-32, которая вычисляется по полезным данным блока.

Как и любая асимметричная криптосистема, система Рабина использует пару ключей. Открытый ключ служит для шифрования и может быть опубликован. Закрытый ключ необходим для расшифрования и должен быть известен только получателю сообщений.

Для генерации ключей выбираются два случайных простых числа p и q , которые должны быть достаточно большими, не равными друг другу и удовлетворять условиям $p=4k+3$; $q=4k+3$ и $p \neq q$, где $k \in \mathbb{N}$. Такие числа называются простыми числами Блюма. В разработанной программе длина каждого из чисел составляет 256 или 512 бит, поэтому модуль n имеет длину примерно 512 или 1024 бита. Простота кандидатов проверяется вероятностным тестом Миллера – Рабина, а нужный вид $4k+3$ достигается прибавлением к случайному нечётному числу небольшой поправки. Затем вычисляется число $n=p \cdot q$. Число n является открытым ключом, а числа p и q – закрытым. В программе ключи сохраняются в текстовые файлы в шестнадцатеричном виде.

Исходное сообщение m , представленное в виде целого числа, шифруется с помощью открытого ключа по формуле $c = m^2 \bmod n$. Чтобы выполнялось условие $m < n$, файл разбивается на блоки, длина которых на один байт меньше длины модуля в байтах.

Для расшифрования необходим закрытый ключ, то есть числа p и q . Так как p и q имеют вид $4k + 3$, квадратные корни из c по каждому из простых модулей находятся по формулам $mp = c^{((p+1)/4)} \bmod p$ и $mq = c^{((q+1)/4)} \bmod q$. Затем с помощью расширенного алгоритма Евклида из уравнения $ur \cdot p + uq \cdot q = 1$ находят числа ur и uq . После этого по китайской теореме об остатках вычисляют четыре числа: $r = (ur \cdot p \cdot mq + uq \cdot q \cdot mp) \bmod n$, $-r = n - r$, $s = (ur \cdot p \cdot mq - uq \cdot q \cdot mp) \bmod n$ и $-s = n - s$. Одно из этих чисел является исходным блоком открытого текста m . Истинный корень определяется проверкой маркера «RBN1» и совпадением контрольной суммы CRC-32.

Задание: изучить теоретические сведения и реализовать криптостойкое программное средство шифрования и дешифрования файлов при помощи криптосистемы Рабина.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм шифрования в криптосистеме Рабина представлен в виде блок-схемы на рисунке 1.1.

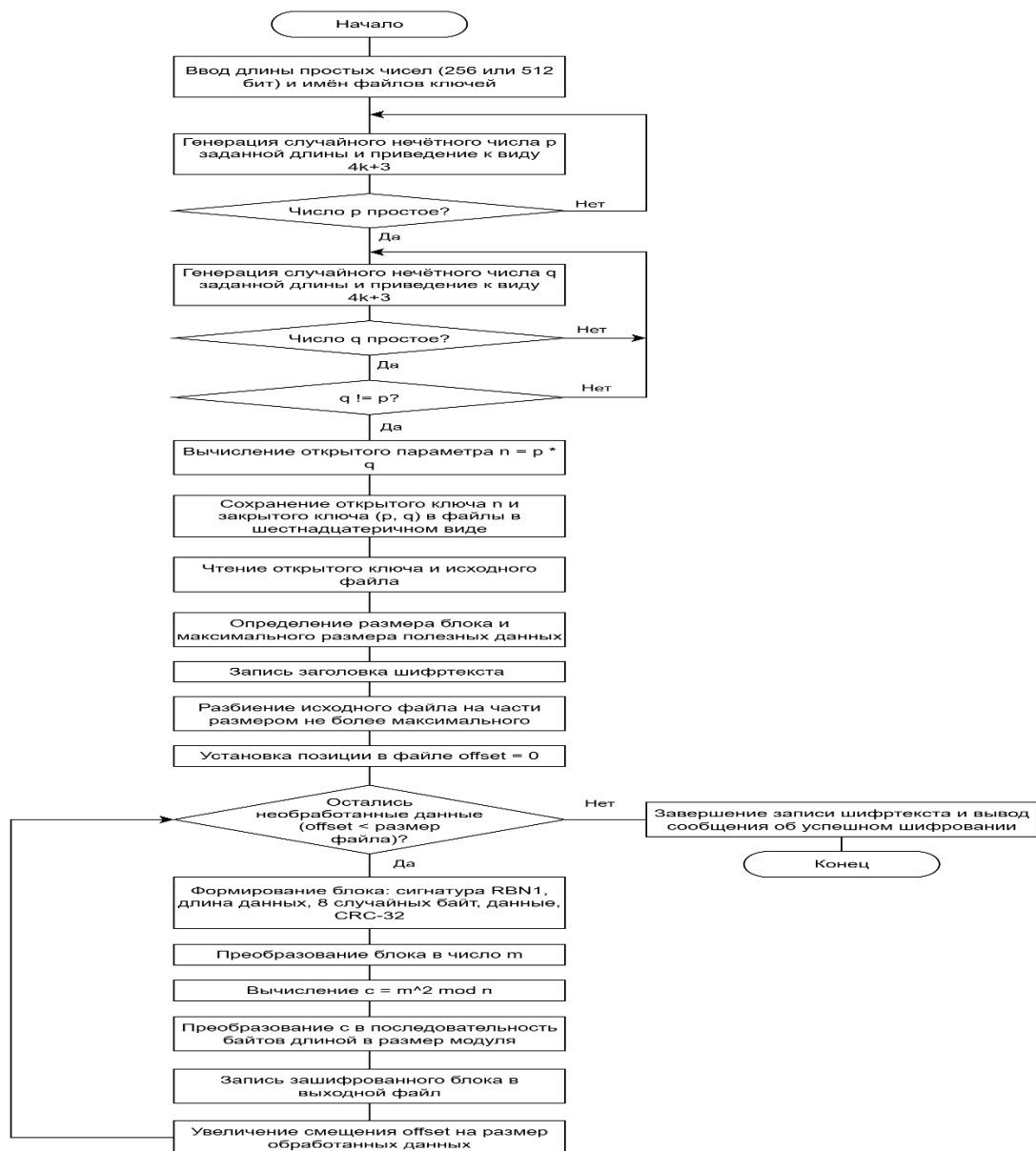


Рисунок 1.1 – Блок-схема алгоритма шифрования в криптосистеме Рабина

Для проверки работы разработанного алгоритма подается файл test.txt, содержимое которого представлено на рисунке 1.2.

```
test.txt
1 Test message for Rabin cryptosystem.
2
```

Рисунок 1.2 – Содержимое тестового файла

Результат генерации открытого и закрытого ключей представлен на рисунке 1.3.

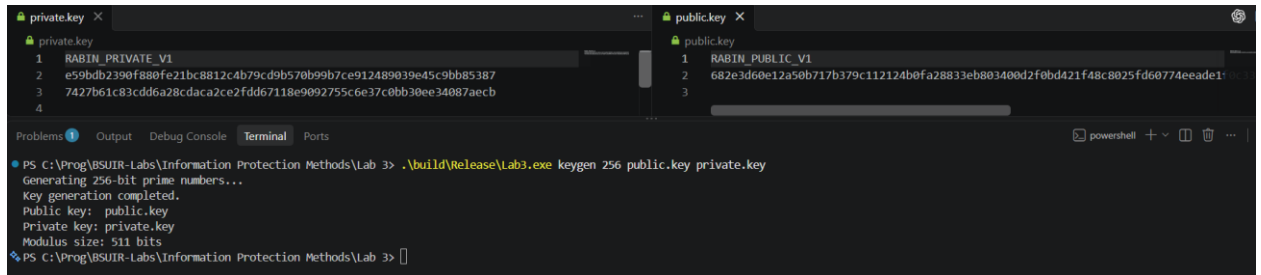


Рисунок 1.3 – Результат генерации открытого и закрытого ключей

Результат выполнения работы программы в режиме шифрования представлен на рисунке 1.4.

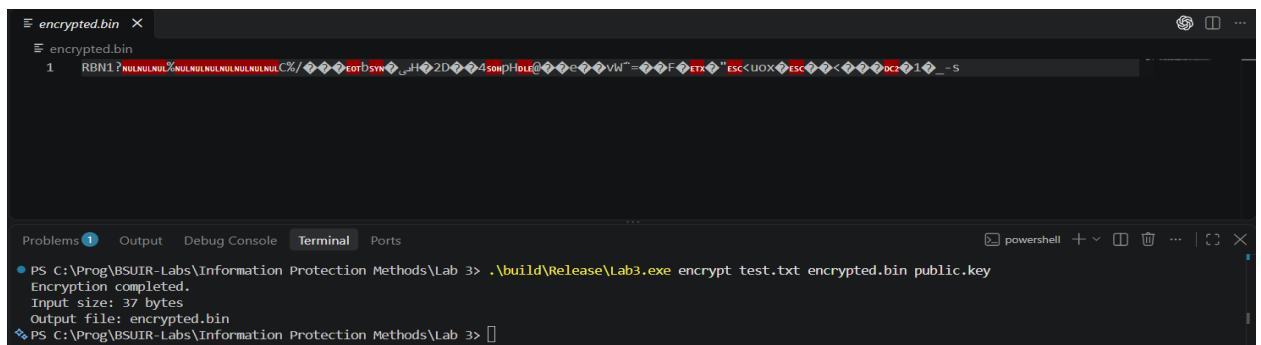


Рисунок 1.4 – Результат выполнения работы программы в режиме шифрования

Результат выполнения работы программы в режиме дешифрования представлен на рисунке 1.5.



Рисунок 1.5 – Результат выполнения работы программы в режиме дешифрования

ВЫВОД

В ходе лабораторной работы была изучена и программно реализована криптосистема Рабина – асимметричная криптосистема, стойкость которой основана на сложности разложения большого числа на простые множители. Были рассмотрены все основные этапы её работы: генерация ключей, шифрование и расшифрование.

В рамках работы разработано консольное программное средство на языке C++, выполняющее генерацию открытого и закрытого ключей на основе простых чисел Блума длиной 256 или 512 бит, шифрование файлов и их последующее расшифрование. Для работы с числами такой разрядности был реализован собственный класс длинной арифметики, а также функции возведения в степень по модулю, проверки простоты числа тестом Миллера – Рабина и расширенного алгоритма Евклида.

Особенностью криптосистемы Рабина является то, что при расшифровании одного блока получается четыре возможных варианта открытого текста. Для однозначного выбора истинного корня в каждый блок перед шифрованием добавляются служебные данные: сигнатура «RBN1», длина полезных данных и контрольная сумма CRC-32. При расшифровании истинным считается тот корень, у которого совпадают сигнатура и контрольная сумма. Такой подход также позволяет обнаружить повреждение шифртекста или использование неверного ключа.

В результате выполнения лабораторной работы была проверена корректность реализованного программного средства на тестовом файле. После шифрования и последующего расшифрования содержимое исходного и восстановленного файлов совпадает.

Таким образом, в ходе лабораторной работы были закреплены практические навыки применения асимметричных криптографических алгоритмов, работы с открытыми и закрытыми ключами, использования расширенного алгоритма Евклида и китайской теоремы об остатках, а также программной обработки файлов с использованием криптосистемы Рабина.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

```
//BigInt.h
#pragma once

#include <vector>
#include <string>
#include <cstdint>
#include <algorithm>
#include <stdexcept>
#include <iostream>

class BigInt {
public:
    BigInt() : digits_{0}, negative_(false) {}

    BigInt(int64_t value) : negative_(value < 0) {
        uint64_t abs_value = negative_ ? -static_cast<uint64_t>(value) :
value;
        if (abs_value == 0) {
            digits_ = {0};
        } else {
            while (abs_value > 0) {
                digits_.push_back(abs_value % BASE);
                abs_value /= BASE;
            }
        }
    }

    BigInt(const std::string& str, int base = 10);

    bool isZero() const {
        return digits_.size() == 1 && digits_[0] == 0;
    }

    bool isNegative() const {
        return negative_ && !isZero();
    }

    BigInt operator+(const BigInt& other) const;
    BigInt operator-(const BigInt& other) const;
    BigInt operator*(const BigInt& other) const;
    BigInt operator/(const BigInt& other) const;
    BigInt operator%(const BigInt& other) const;

    BigInt& operator+=(const BigInt& other);
    BigInt& operator-=(const BigInt& other);
    BigInt& operator*=(const BigInt& other);
    BigInt& operator/=(const BigInt& other);
    BigInt& operator%=(const BigInt& other);

    BigInt& operator<<=(int shift);
    BigInt& operator>>=(int shift);
    BigInt operator<<(int shift) const;
    BigInt operator>>(int shift) const;

    BigInt& operator|=(const BigInt& other);
    BigInt& operator&=(const BigInt& other);
    BigInt operator|(const BigInt& other) const;
    BigInt operator&(const BigInt& other) const;
```

```

    bool operator==(const BigInt& other) const;
    bool operator!=(const BigInt& other) const;
    bool operator<(const BigInt& other) const;
    bool operator<=(const BigInt& other) const;
    bool operator>(const BigInt& other) const;
    bool operator>=(const BigInt& other) const;

    std::string toString(int base = 10) const;
    size_t bitLength() const;
    void setBit(size_t pos);
    bool getBit(size_t pos) const;

    static BigInt fromBytes(const std::vector<uint8_t>& bytes);
    std::vector<uint8_t> toBytes(size_t size) const;

private:
    static constexpr uint32_t BASE = 1000000000;
    static constexpr int BASE_DIGITS = 9;

    std::vector<uint32_t> digits_;
    bool negative_;

    void removeLeadingZeros();
    int compareAbs(const BigInt& other) const;
    BigInt addAbs(const BigInt& other) const;
    BigInt subAbs(const BigInt& other) const;
    void divMod(const BigInt& divisor, BigInt& quotient, BigInt&
remainder) const;
};

BigInt::BigInt(const std::string& str, int base) : negative_(false) {
    if (str.empty()) {
        digits_ = {0};
        return;
    }

    size_t start = 0;
    if (str[0] == '-') {
        negative_ = true;
        start = 1;
    }

    if (base == 16) {
        digits_ = {0};
        for (size_t i = start; i < str.length(); ++i) {
            char c = str[i];
            int digit;
            if (c >= '0' && c <= '9') digit = c - '0';
            else if (c >= 'a' && c <= 'f') digit = c - 'a' + 10;
            else if (c >= 'A' && c <= 'F') digit = c - 'A' + 10;
            else continue;

            *this = *this * BigInt(16) + BigInt(digit);
        }
    } else {
        digits_ = {0};
        for (size_t i = start; i < str.length(); ++i) {
            if (str[i] >= '0' && str[i] <= '9') {
                *this = *this * BigInt(10) + BigInt(str[i] - '0');
            }
        }
    }
}

```

```

        if (isZero()) negative_ = false;
    }

    void BigInt::removeLeadingZeros() {
        while (digits_.size() > 1 && digits_.back() == 0) {
            digits_.pop_back();
        }
        if (isZero()) negative_ = false;
    }

    int BigInt::compareAbs(const BigInt& other) const {
        if (digits_.size() != other.digits_.size()) {
            return digits_.size() < other.digits_.size() ? -1 : 1;
        }
        for (int i = static_cast<int>(digits_.size()) - 1; i >= 0; --i) {
            if (digits_[i] != other.digits_[i]) {
                return digits_[i] < other.digits_[i] ? -1 : 1;
            }
        }
        return 0;
    }

    BigInt BigInt::addAbs(const BigInt& other) const {
        BigInt result;
        result.digits_.clear();
        uint64_t carry = 0;
        size_t maxSize = std::max(digits_.size(), other.digits_.size());

        for (size_t i = 0; i < maxSize || carry; ++i) {
            uint64_t sum = carry;
            if (i < digits_.size()) sum += digits_[i];
            if (i < other.digits_.size()) sum += other.digits_[i];
            result.digits_.push_back(sum % BASE);
            carry = sum / BASE;
        }

        return result;
    }

    BigInt BigInt::subAbs(const BigInt& other) const {
        BigInt result;
        result.digits_.clear();
        int64_t borrow = 0;

        for (size_t i = 0; i < digits_.size(); ++i) {
            int64_t diff = digits_[i] - borrow;
            if (i < other.digits_.size()) diff -= other.digits_[i];

            if (diff < 0) {
                diff += BASE;
                borrow = 1;
            } else {
                borrow = 0;
            }
            result.digits_.push_back(diff);
        }

        result.removeLeadingZeros();
        return result;
    }

    BigInt BigInt::operator+(const BigInt& other) const {
        if (negative_ == other.negative_) {
            BigInt result = addAbs(other);

```

```

        result.negative_ = negative_;
        return result;
    }

    int cmp = compareAbs(other);
    if (cmp == 0) return BigInt(0);

    if (cmp > 0) {
        BigInt result = subAbs(other);
        result.negative_ = negative_;
        return result;
    } else {
        BigInt result = other.subAbs(*this);
        result.negative_ = other.negative_;
        return result;
    }
}

BigInt BigInt::operator-(const BigInt& other) const {
    if (negative_ != other.negative_) {
        BigInt result = addAbs(other);
        result.negative_ = negative_;
        return result;
    }

    int cmp = compareAbs(other);
    if (cmp == 0) return BigInt(0);

    if (cmp > 0) {
        BigInt result = subAbs(other);
        result.negative_ = negative_;
        return result;
    } else {
        BigInt result = other.subAbs(*this);
        result.negative_ = !negative_;
        return result;
    }
}

BigInt BigInt::operator*(const BigInt& other) const {
    BigInt result;
    result.digits_.assign(digits_.size() + other.digits_.size(), 0);

    for (size_t i = 0; i < digits_.size(); ++i) {
        uint64_t carry = 0;
        for (size_t j = 0; j < other.digits_.size() || carry; ++j) {
            uint64_t current = result.digits_[i + j] +
                digits_[i] * 1ULL * (j < other.digits_.size()
? other.digits_[j] : 0) + carry;
            result.digits_[i + j] = current % BASE;
            carry = current / BASE;
        }
    }

    result.removeLeadingZeros();
    result.negative_ = negative_ != other.negative_;
    if (result.isZero()) result.negative_ = false;
    return result;
}

void BigInt::divMod(const BigInt& divisor, BigInt& quotient, BigInt&
remainder) const {
    if (divisor.isZero()) {
        throw std::domain_error("Division by zero");
    }
}

```

```

    }

    quotient = BigInt(0);
    remainder = *this;
    remainder.negative_ = false;

    if (compareAbs(divisor) < 0) {
        quotient = BigInt(0);
        remainder = *this;
        remainder.negative_ = false;
        return;
    }

    BigInt current(0);
    quotient.digits_.clear();

    for (int i = static_cast<int>(digits_.size()) - 1; i >= 0; --i) {
        current = current * BigInt(BASE) +
        BigInt(static_cast<int64_t>(digits_[i]));

        int x = 0;
        int left = 0, right = BASE;
        while (left <= right) {
            int mid = (left + right) / 2;
            BigInt test = divisor * BigInt(mid);
            test.negative_ = false;
            if (test.compareAbs(current) <= 0) {
                x = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        quotient.digits_.push_back(x);
        current = current - divisor * BigInt(x);
        current.negative_ = false;
    }

    std::reverse(quotient.digits_.begin(), quotient.digits_.end());
    quotient.removeLeadingZeros();
    remainder = current;
}

BigInt BigInt::operator/(const BigInt& other) const {
    BigInt quotient, remainder;
    divMod(other, quotient, remainder);
    quotient.negative_ = negative_ != other.negative_ &&
!quotient.isZero();
    return quotient;
}

BigInt BigInt::operator%(const BigInt& other) const {
    BigInt quotient, remainder;
    BigInt abs_this = *this;
    abs_this.negative_ = false;
    BigInt abs_other = other;
    abs_other.negative_ = false;

    abs_this.divMod(abs_other, quotient, remainder);

    if (negative_ && !remainder.isZero()) {
        remainder = abs_other - remainder;
    }
}

```

```

        return remainder;
    }

    BigInt& BigInt::operator+=(const BigInt& other) {
        *this = *this + other;
        return *this;
    }

    BigInt& BigInt::operator-=(const BigInt& other) {
        *this = *this - other;
        return *this;
    }

    BigInt& BigInt::operator*=(const BigInt& other) {
        *this = *this * other;
        return *this;
    }

    BigInt& BigInt::operator/=(const BigInt& other) {
        *this = *this / other;
        return *this;
    }

    BigInt& BigInt::operator%=(const BigInt& other) {
        *this = *this % other;
        return *this;
    }

    BigInt& BigInt::operator<<=(int shift) {
        *this = *this << shift;
        return *this;
    }

    BigInt& BigInt::operator>>=(int shift) {
        *this = *this >> shift;
        return *this;
    }

    BigInt BigInt::operator<<(int shift) const {
        if (shift == 0) return *this;
        if (isZero()) return *this;

        BigInt power(1);
        BigInt two(2);

        int s = shift;
        BigInt base(2);
        while (s > 0) {
            if ((s % 2) == 1) {
                power = power * base;
            }
            if (s > 1) {
                base = base * base;
            }
            s = s / 2;
        }

        BigInt result = *this * power;
        result.negative_ = negative_;
        return result;
    }

    BigInt BigInt::operator>>(int shift) const {

```

```

        if (shift == 0) return *this;
        if (isZero()) return *this;

        BigInt power(1);
        BigInt two(2);

        int s = shift;
        BigInt base(2);
        while (s > 0) {
            if ((s % 2) == 1) {
                power = power * base;
            }
            if (s > 1) {
                base = base * base;
            }
            s = s / 2;
        }

        BigInt result = *this / power;
        result.negative_ = negative_;
        return result;
    }

    BigInt& BigInt::operator|=(const BigInt& other) {
        *this = *this | other;
        return *this;
    }

    BigInt& BigInt::operator&=(const BigInt& other) {
        *this = *this & other;
        return *this;
    }

    BigInt BigInt::operator|(const BigInt& other) const {
        size_t maxBits = std::max(bitLength(), other.bitLength());
        BigInt result(0);

        for (size_t i = 0; i < maxBits; ++i) {
            if (getBit(i) || other.getBit(i)) {
                result.setBit(i);
            }
        }

        return result;
    }

    BigInt BigInt::operator&(const BigInt& other) const {
        size_t maxBits = std::max(bitLength(), other.bitLength());
        BigInt result(0);

        for (size_t i = 0; i < maxBits; ++i) {
            if (getBit(i) && other.getBit(i)) {
                result.setBit(i);
            }
        }

        return result;
    }

    bool BigInt::operator==(const BigInt& other) const {
        return negative_ == other.negative_ && digits_ == other.digits_;
    }

    bool BigInt::operator!=(const BigInt& other) const {

```

```

        return !(*this == other);
    }

    bool BigInt::operator<(const BigInt& other) const {
        if (negative_ != other.negative_) {
            return negative_;
        }

        int cmp = compareAbs(other);
        return negative_ ? cmp > 0 : cmp < 0;
    }

    bool BigInt::operator<=(const BigInt& other) const {
        return *this < other || *this == other;
    }

    bool BigInt::operator>(const BigInt& other) const {
        return !(*this <= other);
    }

    bool BigInt::operator>=(const BigInt& other) const {
        return !(*this < other);
    }

    std::string BigInt::toString(int base) const {
        if (isZero()) return "0";

        if (base == 16) {
            std::string result;
            BigInt temp = *this;
            temp.negative_ = false;

            while (!temp.isZero()) {
                int digit = (temp % BigInt(16)).digits_[0];
                result += (digit < 10) ? ('0' + digit) : ('a' + digit - 10);
                temp /= BigInt(16);
            }

            if (negative_) result += '-';
            std::reverse(result.begin(), result.end());
            return result;
        }

        std::string result;
        BigInt temp = *this;
        temp.negative_ = false;

        while (!temp.isZero()) {
            result += '0' + (temp % BigInt(10)).digits_[0];
            temp /= BigInt(10);
        }

        if (negative_) result += '-';
        std::reverse(result.begin(), result.end());
        return result;
    }

    size_t BigInt::bitLength() const {
        if (isZero()) return 1;

        BigInt temp = *this;
        temp.negative_ = false;
        size_t bits = 0;

```

```

        while (!temp.isZero()) {
            temp = temp >> 1;
            bits++;
        }

        return bits;
    }

    void BigInt::setBit(size_t pos) {
        size_t bitsPerDigit = 30;
        size_t digitIndex = pos / bitsPerDigit;
        size_t bitInDigit = pos % bitsPerDigit;

        while (digits_.size() <= digitIndex) {
            digits_.push_back(0);
        }

        int64_t bitValue = 1LL << bitInDigit;
        digits_[digitIndex] |= bitValue;

        if (digits_[digitIndex] >= BASE) {
            int64_t carry = digits_[digitIndex] / BASE;
            digits_[digitIndex] %= BASE;
            if (digitIndex + 1 < digits_.size()) {
                digits_[digitIndex + 1] += carry;
            } else {
                digits_.push_back(carry);
            }
        }
    }

    bool BigInt::getBit(size_t pos) const {
        size_t bitsPerDigit = 30;
        size_t digitIndex = pos / bitsPerDigit;
        size_t bitInDigit = pos % bitsPerDigit;

        if (digitIndex >= digits_.size()) {
            return false;
        }

        int64_t bitValue = 1LL << bitInDigit;
        return (digits_[digitIndex] & bitValue) != 0;
    }

    BigInt BigInt::fromBytes(const std::vector<uint8_t>& bytes) {
        BigInt result(0);

        for (const uint8_t byte : bytes) {
            result = result << 8;
            result = result + BigInt(byte);
        }

        return result;
    }

    std::vector<uint8_t> BigInt::toBytes(size_t size) const {
        std::vector<uint8_t> bytes(size, 0);
        BigInt temp = *this;
        temp.negative_ = false;

        for (size_t i = size; i-- > 0;) {
            bytes[i] = (temp % BigInt(256)).digits_[0];
            temp = temp / BigInt(256);
        }
    }

```

```

    }

    return bytes;
}

//main.cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <random>
#include <cstdint>
#include <algorithm>
#include <stdexcept>

#include "BigInt.h"
#include "PublicKey.h"
#include "PrivateKey.h"

using Byte = std::uint8_t;
using ByteVector = std::vector<Byte>;

std::mt19937_64& getRandomEngine() {
    static std::mt19937_64 generator(std::random_device{}());
    return generator;
}

Byte randomByte() {
    return static_cast<Byte>(getRandomEngine()() & 0xFF);
}

BigInt randomBits(unsigned bitCount) {
    BigInt value = 0;
    const unsigned fullBytes = bitCount / 8;
    const unsigned remainingBits = bitCount % 8;

    for (unsigned i = 0; i < fullBytes; ++i) {
        value <<= 8;
        value += BigInt(randomByte());
    }

    if (remainingBits != 0) {
        value <<= remainingBits;
        Byte lastByte = randomByte();
        lastByte &= (1 << remainingBits) - 1;
        value += BigInt(lastByte);
    }

    value.setBit(bitCount - 1);
    value.setBit(0);

    return value;
}

BigInt modPow(BigInt base, BigInt exponent, const BigInt& modulus) {
    BigInt result = 1;
    base = base % modulus;

    while (exponent > BigInt(0)) {
        if ((exponent & BigInt(1)) != BigInt(0)) {
            result = (result * base) % modulus;
        }
    }
}

```

```

        base = (base * base) % modulus;
        exponent >>= 1;
    }

    return result;
}

BigInt gcd(BigInt a, BigInt b) {
    while (b != BigInt(0)) {
        BigInt remainder = a % b;
        a = b;
        b = remainder;
    }

    return a;
}

BigInt extendedGcd(const BigInt& a, const BigInt& b, BigInt& x, BigInt&
y) {
    if (b == BigInt(0)) {
        x = 1;
        y = 0;
        return a;
    }

    BigInt x1, y1;
    BigInt result = extendedGcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return result;
}

bool isProbablePrime(const BigInt& value, int rounds = 32) {
    static constexpr int smallPrimes[] = {
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
    };

    for (const int prime : smallPrimes) {
        if (value == BigInt(prime)) {
            return true;
        }

        if ((value % BigInt(prime)) == BigInt(0)) {
            return false;
        }
    }

    if (value < BigInt(2) || (value & BigInt(1)) == BigInt(0)) {
        return false;
    }

    BigInt d = value - BigInt(1);
    unsigned powerOfTwo = 0;

    while ((d & BigInt(1)) == BigInt(0)) {
        d >>= 1;
        ++powerOfTwo;
    }

    for (int round = 0; round < rounds; ++round) {
        BigInt base = BigInt(2) + BigInt(getRandomEngine()() % 100);
        if (base >= value - BigInt(2)) {

```

```

        base = BigInt(2);
    }

    BigInt x = modPow(base, d, value);

    if (x == BigInt(1) || x == value - BigInt(1)) {
        continue;
    }

    bool passed = false;

    for (unsigned i = 1; i < powerOfTwo; ++i) {
        x = (x * x) % value;

        if (x == value - BigInt(1)) {
            passed = true;
            break;
        }
    }

    if (!passed) {
        return false;
    }
}

return true;
}

BigInt generateBlumPrime(unsigned bitCount) {
    while (true) {
        BigInt candidate = randomBits(bitCount);

        BigInt remainder = candidate % BigInt(4);
        if (remainder == BigInt(1)) {
            candidate = candidate + BigInt(2);
        } else if (remainder == BigInt(0) || remainder == BigInt(2)) {
            candidate = candidate + BigInt(3);
        }

        if (isProbablePrime(candidate)) {
            return candidate;
        }
    }
}

BigInt bytesToInteger(const ByteVector& bytes) {
    BigInt value = 0;

    for (const Byte byte : bytes) {
        value <<= 8;
        value += BigInt(byte);
    }

    return value;
}

ByteVector integerToBytes(BigInt value, std::size_t size) {
    ByteVector bytes(size, 0);

    for (std::size_t i = size; i-- > 0;) {
        BigInt remainder = value % BigInt(256);
        bytes[i] = static_cast<Byte>(std::stoi(remainder.toString()));
        value = value / BigInt(256);
    }
}

```

```

        return bytes;
    }

    void writeUInt32(std::ostream& stream, std::uint32_t value) {
        for (int i = 0; i < 4; ++i) {
            stream.put(static_cast<char>((value >> (i * 8)) & 0xFF));
        }
    }

    std::uint32_t readUInt32(std::istream& stream) {
        std::uint32_t value = 0;

        for (int i = 0; i < 4; ++i) {
            const int byte = stream.get();

            if (byte == EOF) {
                throw std::runtime_error("Unexpected end of file.");
            }

            value |= static_cast<std::uint32_t>(static_cast<Byte>(byte)) <<
(i * 8);
        }

        return value;
    }

    std::uint64_t readUInt64(std::istream& stream) {
        std::uint64_t value = 0;

        for (int i = 0; i < 8; ++i) {
            const int byte = stream.get();

            if (byte == EOF) {
                throw std::runtime_error("Unexpected end of file.");
            }

            value |= static_cast<std::uint64_t>(static_cast<Byte>(byte)) <<
(i * 8);
        }

        return value;
    }

    void writeUInt64(std::ostream& stream, std::uint64_t value) {
        for (int i = 0; i < 8; ++i) {
            stream.put(static_cast<char>((value >> (i * 8)) & 0xFF));
        }
    }

    std::uint32_t calculateCrc32(const ByteVector& data) {
        std::uint32_t crc = 0xFFFFFFFFu;

        for (const Byte byte : data) {
            crc ^= byte;

            for (int i = 0; i < 8; ++i) {
                const std::uint32_t mask = -(crc & 1u);
                crc = (crc >> 1) ^ (0xEDB88320u & mask);
            }
        }

        return ~crc;
    }

```

```

    void generateKeys(unsigned primeBitCount, const std::string& publicPath,
const std::string& privatePath) {
        std::cout << "Generating " << primeBitCount << "-bit prime
numbers...\n";

        BigInt p = generateBlumPrime(primeBitCount);
        BigInt q;

        do {
            q = generateBlumPrime(primeBitCount);
        } while (p == q);

        PublicKey publicKey;
        publicKey.n = p * q;

        PrivateKey privateKey(p, q);

        publicKey.save(publicPath);
        privateKey.save(privatePath);

        const unsigned modulusBits = publicKey.n.bitLength();

        std::cout << "Key generation completed.\n";
        std::cout << "Public key: " << publicPath << '\n';
        std::cout << "Private key: " << privatePath << '\n';
        std::cout << "Modulus size: " << modulusBits << " bits\n";
    }

constexpr std::size_t BLOCK_HEADER_SIZE = 16;
constexpr std::size_t BLOCK_CRC_SIZE = 4;
constexpr char BLOCK_MAGIC[] = {'R', 'B', 'N', '1'};

    ByteVector encodeBlock(const ByteVector& data, std::size_t offset,
std::size_t size, std::size_t blockSize) {
        ByteVector block(blockSize, 0);

        std::copy(std::begin(BLOCK_MAGIC), std::end(BLOCK_MAGIC),
block.begin());

        const auto dataSize = static_cast<std::uint32_t>(size);

        for (int i = 0; i < 4; ++i) {
            block[4 + i] = static_cast<Byte>((dataSize >> (i * 8)) & 0xFF);
        }

        for (int i = 0; i < 8; ++i) {
            block[8 + i] = randomByte();
        }

        std::copy(data.begin() + static_cast<std::ptrdiff_t>(offset),
data.begin() + static_cast<std::ptrdiff_t>(offset + size),
block.begin() + BLOCK_HEADER_SIZE);

        ByteVector payload(
            data.begin() + static_cast<std::ptrdiff_t>(offset),
            data.begin() + static_cast<std::ptrdiff_t>(offset + size)
        );

        const std::uint32_t checksum = calculateCrc32(payload);
        const std::size_t checksumOffset = BLOCK_HEADER_SIZE + size;

        for (int i = 0; i < 4; ++i) {
            block[checksumOffset + i] =

```

```

        static_cast<Byte>((checksum >> (i * 8)) & 0xFF);
    }

    return block;
}

bool decodeBlock(const ByteVector& block, ByteVector& data) {
    if (block.size() < BLOCK_HEADER_SIZE + BLOCK_CRC_SIZE) {
        return false;
    }

    if (!std::equal(std::begin(BLOCK_MAGIC), std::end(BLOCK_MAGIC),
block.begin())) {
        return false;
    }

    std::uint32_t dataSize = 0;

    for (int i = 0; i < 4; ++i) {
        dataSize |= static_cast<std::uint32_t>(block[4 + i]) << (i * 8);
    }

    if (dataSize > block.size() - BLOCK_HEADER_SIZE - BLOCK_CRC_SIZE) {
        return false;
    }

    const std::size_t checksumOffset = BLOCK_HEADER_SIZE + dataSize;
    std::uint32_t storedChecksum = 0;

    for (int i = 0; i < 4; ++i) {
        storedChecksum |=
            static_cast<std::uint32_t>(block[checksumOffset + i]) << (i
* 8);
    }

    data.assign(
        block.begin() + BLOCK_HEADER_SIZE,
        block.begin() + BLOCK_HEADER_SIZE + dataSize
    );

    return calculateCrc32(data) == storedChecksum;
}

BigInt rabinEncrypt(const BigInt& message, const PublicKey& key) {
    return (message * message) % key.n;
}

ByteVector rabinDecrypt(const BigInt& ciphertext, const PrivateKey& key,
std::size_t blockSize) {
    BigInt pRoot = modPow(
        ciphertext % key.p,
        (key.p + BigInt(1)) / BigInt(4),
        key.p
    );

    BigInt qRoot = modPow(
        ciphertext % key.q,
        (key.q + BigInt(1)) / BigInt(4),
        key.q
    );

    BigInt pCoefficient, qCoefficient;
    extendedGcd(key.p, key.q, pCoefficient, qCoefficient);

```

```

        BigInt root1 =
            (pCoefficient * key.p * qRoot +
             qCoefficient * key.q * pRoot) %
            key.n;

        if (root1 < BigInt(0)) {
            root1 += key.n;
        }

        BigInt root2 = key.n - root1;

        BigInt root3 =
            (pCoefficient * key.p * qRoot -
             qCoefficient * key.q * pRoot) %
            key.n;

        if (root3 < BigInt(0)) {
            root3 += key.n;
        }

        BigInt root4 = key.n - root3;

        const std::vector<BigInt> roots = {root1, root2, root3, root4};

        for (const BigInt& root : roots) {
            ByteVector block = integerToBytes(root, blockSize);
            ByteVector data;

            if (decodeBlock(block, data)) {
                return block;
            }
        }

        throw std::runtime_error(
            "No valid plaintext root was found. The key or ciphertext may be
            corrupted."
        );
    }

    ByteVector readFile(const std::string& path) {
        std::ifstream input(path, std::ios::binary);

        if (!input) {
            throw std::runtime_error("Cannot open input file.");
        }

        return ByteVector(
            std::istreambuf_iterator<char>(input),
            std::istreambuf_iterator<char>()
        );
    }

    void writeEncryptedBlock(std::ofstream& output, const ByteVector& block,
        const PublicKey& key, std::size_t modulusBytes) {
        const BigInt message = bytesToInteger(block);
        const BigInt ciphertext = rabinEncrypt(message, key);

        const ByteVector encrypted = integerToBytes(ciphertext,
            modulusBytes);

        output.write(
            reinterpret_cast<const char*>(encrypted.data()),
            static_cast<std::streamsize>(encrypted.size())
        );
    }

```

```

    }

    void encryptFile(const std::string& inputPath, const std::string&
outputPath, const std::string& publicKeyPath) {
        const PublicKey key = PublicKey::load(publicKeyPath);
        const ByteVector data = readFile(inputPath);

        const std::size_t modulusBytes = (key.n.bitLength() + 7) / 8;
        const std::size_t blockSize = modulusBytes - 1;

        if (blockSize <= BLOCK_HEADER_SIZE + BLOCK_CRC_SIZE) {
            throw std::runtime_error("The modulus is too small.");
        }

        const std::size_t maxPayloadSize = blockSize - BLOCK_HEADER_SIZE -
BLOCK_CRC_SIZE;

        std::ofstream output(outputPath, std::ios::binary);

        if (!output) {
            throw std::runtime_error("Cannot create ciphertext file.");
        }

        output.write("RBN1", 4);
        writeUInt32(output, static_cast<std::uint32_t>(blockSize));
        writeUInt64(output, static_cast<std::uint64_t>(data.size()));

        if (data.empty()) {
            const ByteVector block = encodeBlock(data, 0, 0, blockSize);
            writeEncryptedBlock(output, block, key, modulusBytes);
        } else {
            for (std::size_t offset = 0; offset < data.size(); ) {
                const std::size_t payloadSize = std::min(maxPayloadSize,
data.size() - offset);

                const ByteVector block = encodeBlock(data, offset, payloadSize,
blockSize);

                writeEncryptedBlock(output, block, key, modulusBytes);
                offset += payloadSize;
            }

            std::cout << "Encryption completed.\n";
            std::cout << "Input size: " << data.size() << " bytes\n";
            std::cout << "Output file: " << outputPath << '\n';
        }
    }

    void decryptFile(const std::string& inputPath, const std::string&
outputPath, const std::string& privateKeyPath) {
        const PrivateKey key = PrivateKey::load(privateKeyPath);

        std::ifstream input(inputPath, std::ios::binary);

        if (!input) {
            throw std::runtime_error("Cannot open ciphertext file.");
        }

        char header[4];
        input.read(header, 4);

        if (input.gcount() != 4 || std::string(header, 4) != "RBN1") {
            throw std::runtime_error("Invalid ciphertext format.");
        }
    }

```

```

const std::uint32_t blockSize = readUInt32(input);
const std::uint64_t originalSize = readUInt64(input);

const std::size_t modulusBytes = (key.n.bitLength() + 7) / 8;

if (blockSize != modulusBytes - 1) {
    throw std::runtime_error("Ciphertext and private key do not
match.");
}

std::ofstream output(outputPath, std::ios::binary);

if (!output) {
    throw std::runtime_error("Cannot create decrypted file.");
}

std::uint64_t written = 0;

while (input.peek() != EOF) {
    ByteVector encryptedBlock(modulusBytes);
    input.read(
        reinterpret_cast<char*>(encryptedBlock.data()),
        static_cast<std::streamsize>(encryptedBlock.size())
    );

    if (input.gcount() !=
static_cast<std::streamsize>(encryptedBlock.size())) {
        throw std::runtime_error("Incomplete ciphertext block.");
    }

    const BigInt ciphertext = bytesToInteger(encryptedBlock);

    if (ciphertext >= key.n) {
        throw std::runtime_error("Invalid ciphertext block.");
    }

    const ByteVector block = rabinDecrypt(ciphertext, key, blockSize);

    ByteVector plaintext;

    if (!decodeBlock(block, plaintext)) {
        throw std::runtime_error("Invalid plaintext block.");
    }

    const std::uint64_t remaining = originalSize - written;
    const std::size_t bytesToWrite =
        static_cast<std::size_t>(
            std::min<std::uint64_t>(plaintext.size(), remaining)
        );

    output.write(
        reinterpret_cast<const char*>(plaintext.data()),
        static_cast<std::streamsize>(bytesToWrite)
    );

    written += bytesToWrite;
}

if (written != originalSize) {
    throw std::runtime_error(
        "Decrypted file size does not match the original size."
    );
}

```

```

        std::cout << "Decryption completed.\n";
        std::cout << "Output size: " << written << " bytes\n";
        std::cout << "Output file: " << outputPath << '\n';
    }

    int main(int argc, char* argv[]) {
        if (argc < 2) {
            std::cerr << "Usage:\n";
            std::cerr << "  Generate keys: " << argv[0] << " keygen <bits>
<public_key> <private_key>\n";
            std::cerr << "  Encrypt file: " << argv[0] << " encrypt <input>
<output> <public_key>\n";
            std::cerr << "  Decrypt file: " << argv[0] << " decrypt <input>
<output> <private_key>\n";
            return 1;
        }

        try {
            std::string mode = argv[1];

            if (mode == "keygen") {
                if (argc != 5) {
                    std::cerr << "Usage: " << argv[0] << " keygen <bits>
<public_key> <private_key>\n";
                    return 1;
                }

                unsigned bits = std::stoi(argv[2]);
                std::string publicPath = argv[3];
                std::string privatePath = argv[4];

                if (bits != 256 && bits != 512) {
                    std::cerr << "Error: only 256 or 512 bits are supported\n";
                    return 1;
                }

                generateKeys(bits, publicPath, privatePath);
            }
            else if (mode == "encrypt") {
                if (argc != 5) {
                    std::cerr << "Usage: " << argv[0] << " encrypt <input>
<output> <public_key>\n";
                    return 1;
                }

                std::string inputPath = argv[2];
                std::string outputPath = argv[3];
                std::string keyPath = argv[4];

                encryptFile(inputPath, outputPath, keyPath);
            }
            else if (mode == "decrypt") {
                if (argc != 5) {
                    std::cerr << "Usage: " << argv[0] << " decrypt <input>
<output> <private_key>\n";
                    return 1;
                }

                std::string inputPath = argv[2];
                std::string outputPath = argv[3];
                std::string keyPath = argv[4];

                decryptFile(inputPath, outputPath, keyPath);
            }
        }
    }

```

```

    }
    else {
        std::cerr << "Error: unknown mode '" << mode << "'\n";
        std::cerr << "Valid modes: keygen, encrypt, decrypt\n";
        return 1;
    }
}
catch (const std::exception& e) {
    std::cerr << "Error: " << e.what() << '\n';
    return 1;
}

return 0;
}

//PrivateKey.cpp
#pragma once

#include "BigInt.h"
#include <string>
#include <fstream>
#include <stdexcept>
#include <utility>

class PrivateKey {
public:
    BigInt p;
    BigInt q;
    BigInt n;

    PrivateKey() = default;

    PrivateKey(BigInt p_, BigInt q_)
        : p(std::move(p_)), q(std::move(q_)), n(p * q) {}

    void save(const std::string& path) const {
        std::ofstream output(path);

        if (!output) {
            throw std::runtime_error("Cannot create private key file.");
        }

        output << "RABIN_PRIVATE_V1\n";
        output << p.toString(16) << '\n';
        output << q.toString(16) << '\n';
    }

    static PrivateKey load(const std::string& path) {
        std::ifstream input(path);

        if (!input) {
            throw std::runtime_error("Cannot open private key file.");
        }

        std::string header;
        input >> header;

        if (header != "RABIN_PRIVATE_V1") {
            throw std::runtime_error("Invalid private key format.");
        }

        std::string pHex, qHex;
        input >> pHex >> qHex;
    }
};

```

```

        return PrivateKey(BigInt(pHex, 16), BigInt(qHex, 16));
    }
};

//PublicKey.h
#pragma once

#include "BigInt.h"
#include <string>
#include <fstream>
#include <stdexcept>

class PublicKey {
public:
    BigInt n;

    PublicKey() = default;

    void save(const std::string& path) const {
        std::ofstream output(path);

        if (!output) {
            throw std::runtime_error("Cannot create public key file.");
        }

        output << "RABIN_PUBLIC_V1\n";
        output << n.toString(16) << '\n';
    }

    static PublicKey load(const std::string& path) {
        std::ifstream input(path);

        if (!input) {
            throw std::runtime_error("Cannot open public key file.");
        }

        std::string header;
        input >> header;

        if (header != "RABIN_PUBLIC_V1") {
            throw std::runtime_error("Invalid public key format.");
        }

        std::string hexModulus;
        input >> hexModulus;

        PublicKey key;
        key.n = BigInt(hexModulus, 16);
        return key;
    }
};

```